

Mamba Sequence Modeling meets Model Predictive Control

M. Ceva¹, T.O. de Jong² (Student Member, IEEE), M. Lazar² (Senior Member, IEEE)

¹Tilburg University, The Netherlands

²Eindhoven University of Technology, The Netherlands

CORRESPONDING AUTHOR: M. Ceva (e-mail: m.cevaal@tilburguniversity.edu)

ABSTRACT In this paper, we consider the design of Model Predictive Control (MPC) algorithms based on Mamba neural networks. Mamba is a neural network architecture capable of sub-quadratic computational scaling in sequence length with state-of-the-art modeling capabilities. We provide a consistent and complete mathematical description of the Mamba neural network. Then, adjustments and optimizations are made to construct a decoder-only Mamba multi-step predictor for MPC and an input-output formulation is given for sequence-to-sequence modeling of dynamical systems. The performance of Mamba-MPC is evaluated on several numerical examples and compared to a Long-Short-Term-Memory-Based MPC (LSTM-MPC) and a Transformer based MPC (Transformer-MPC) for computational scaling comparison. First, a Single-Input-Single-Output (SISO) Van der Pol oscillator is considered, where stability, reference tracking, and noise robustness are evaluated. Then, a MIMO Four Tank setup is introduced where Multiple-Input-Multiple-Output (MIMO) reference tracking is evaluated. Lastly, Mamba-MPC is implemented on a physical Quanser Aero2 setup for closed-loop reference tracking. The results demonstrate that Mamba-MPC is able to stabilize and track a reference for SISO and MIMO systems, both in simulation and on a physical setup. Moreover, Mamba-MPC consistently outperforms LSTM-MPC in predictive control and is significantly computationally faster.

INDEX TERMS Model predictive control, Neural networks, Recurrent neural networks, Machine learning, Data-driven modeling

I. INTRODUCTION

Model Predictive Control (MPC) is a highly successful advanced control methodology which can be merged with *Neural Networks* (NN) as predictors and thus leverage their universal approximation properties [1] for closed-loop control. Particularly when operating on complex nonlinear systems, this is an interesting approach because accurate first-principles models are often difficult to obtain. Nevertheless, a crucial limiting factor for the successful application of NN within the MPC framework is that their intricate nonlinear architecture inherently introduces computational complexity, which restricts computational tractability and poses challenges for online implementation. Early efforts to model nonlinear dynamics for MPC used feedforward NNs [2], but these models struggle to capture long-range temporal dependencies and suffer from cascading errors in multi-step predictions. Recurrent architectures such as *Recurrent Neural Networks* (RNN) for MPC [3] address this by propagating temporal information via latent states and have been shown to explicitly exhibit state-space dynamics in [4]. Nevertheless, modeling long-range dependencies with RNN remains a challenge. Consequently, gating augmentations

were introduced in *Long Short-Term Memory* (LSTM) [5], [6] and *Gated Recurrent Units* (GRU) [7], [8] to enable long-range prediction. Despite these advances, accurate modeling over longer prediction horizons remains challenging due to the persistent accumulation of errors inherent to one-step ahead predictors. More recently, Transformer networks [9] have emerged for modeling global temporal relations which is particularly advantageous in the modeling of extremely long prediction horizons. A multi-step Transformer predictor for MPC was constructed in [10], and compared to LSTM. In comparison to LSTM, the multi-step Transformer showed superior performance and speed, although the computational benefit largely stemmed from the construction of a multi-step predictor. A challenge with Transformers is that the main mechanism of Transformers, attention, leads to quadratic computational scaling when increasing the length of the prediction horizon. This limits the practical implementation of Transformers in MPC. A novel NN architecture, Mamba [11], has recently emerged as a compelling alternative, built on structured state-space models [12]. Mamba incorporates a context-aware selection mechanism that propagates relevant information from past sequence steps in the hidden states,

effectively incorporating the gating mechanism found in GRUs. Gating mechanisms in NNs have been shown to enable the network to learn time warping functions [13], which aids efficient storing of information. Crucially, the selection mechanism enables Mamba to scale linearly in computational cost when increasing the sequence length. Furthermore, Mamba has demonstrated superior performance compared to Transformers and DeepONet [14] in the modeling of dynamical systems [15] when used as a neural operator for learning *Sequence-to-Sequence* (Seq2Seq) mappings. Consequently, a Mamba-based prediction model is an excellent candidate for predictive control, which, to the best of our knowledge, has not been explored.

It is worth mentioning that an approach that combines elements of the Mamba architecture with a linear-in-control Koopman operator approach for MPC was recently presented in [16]. Therein Mamba is used as a linear state-space matrix generator in combination with a Koopman operator to form a lifted time-varying linear state-space system model. To the best of our knowledge, an implementation of Mamba for MPC where the full Mamba architecture is employed as a multi-step predictor has not yet been explored.

Therefore, in this paper, we investigate the application of Mamba in predictive control of nonlinear dynamical systems by constructing a data-driven Mamba-MPC algorithm. The first challenge is to construct a Mamba-based multi-step predictor while maintaining equal sequence length between input and output. The second challenge is to construct a multi-step formulation of Seq2Seq to enable input/output mapping for a set of initial conditions. We thus aim to formulate a Mamba-based predictor that allows for MIMO multi-step predictions. The first contribution of this paper is (i) a complete and consistent mathematical description of the underlying operations employed within the Mamba NN architecture. Note that, to the best of our knowledge, a complete rigorous mathematical description of the Mamba NN is not available in the literature, which currently hampers understanding of the underlying functional operations and reproducibility. Furthermore, we include a Lipschitz boundedness analysis of the complete Mamba neural network architecture. The second contribution is (ii) to provide an adaptation and optimization of Mamba for Mamba-MPC as well as a Seq2Seq input-output embedding. The third contribution is (iii) implementation of Mamba-MPC in real-time control of a challenging physical Quanser Aero2 system. To enhance reproducibility, we provide for all learning and MPC algorithms the implementation code in Python using PyTorch and CasADI.

The remainder of the paper is structured as follows. Section II introduces notations and definitions. Section III reviews MPC preliminaries and define the problem statement. In Section IV a mathematical description of Mamba is provided. In Section V, the Mamba-MPC algorithm is derived. In Section VI, we test Mamba-MPC for 2 numerical examples, the Van der Pol oscillator and a MIMO Four Tank

system. In Section VII we provide an implementation on a physical setup as a proof of concept for practical application of Mamba-MPC. Section VIII presents conclusions and future work.

II. Notations and Definitions

Basic notation and definitions are given next.

a: Vectors

For a finite collection of vectors $\xi_i \in \mathbb{R}^{n_i}$ ($i = 1, \dots, q$), let $\text{col}(\xi_1, \dots, \xi_q) := [\xi_1^\top \dots \xi_q^\top]^\top$.

b: Matrices

For any two matrices $A \in \mathbb{R}^{n \times m}$ $B \in \mathbb{R}^{n \times m}$, we denote the Hadamard product by $A \odot B$, defined as

$$A \odot B = \begin{bmatrix} a_{1,1}b_{1,1} & \dots & a_{1,m}b_{1,m} \\ \vdots & & \vdots \\ a_{n,1}b_{n,1} & \dots & a_{n,m}b_{n,m} \end{bmatrix}$$

which can be extended to high-order tensors. Given scalars $a_1, a_2, \dots, a_n \in \mathbb{R}$ we define the diagonal matrix $A = \text{diag}(a_1, a_2, \dots, a_n) \in \mathbb{R}^{n \times n}$ where $A_{i,j} = a_i$ when $i = j$ and $A_{i,j} = 0$ when $i \neq j$ such that A has $a_1, a_2, \dots, a_n$ on its diagonal and zeros elsewhere. Given a vector $w = [w_1, \dots, w_l]^\top \in \mathbb{R}^l$ we define a Toeplitz matrix constructed from vector w as

$$\text{Toeplitz}(w) = \begin{bmatrix} w_1 & \dots & w_l & 0 & \dots & 0 \\ 0 & w_1 & \dots & w_l & \dots & 0 \\ \vdots & \vdots & \ddots & \ddots & \ddots & \vdots \\ 0 & \dots & 0 & w_1 & \dots & w_l \end{bmatrix}.$$

c: Tensors

In this paper tensors up to rank 3 are mostly considered. Let calligraphic variables, e.g., $\mathcal{T} \in \mathbb{R}^{d_1 \times d_2 \times d_3}$ denote a tensor. When operating on a tensor, let $\mathcal{T}_{:,i,:} \in \mathbb{R}^{d_1 \times d_3}$, denote the i^{th} slice of the second dimension, e.g., for a tensor $\mathcal{B} \in \mathbb{R}^{2 \times 2 \times 2}$, e.g.,

$$\mathcal{B} = \left(\left\{ \begin{bmatrix} b_1 & b_2 \\ b_3 & b_4 \end{bmatrix}, \begin{bmatrix} b_5 & b_6 \\ b_7 & b_8 \end{bmatrix} \right\} \right),$$

we denote the 1-st slice of dimension d_2 by:

$$\mathcal{B}_{:,1,:} = \left\{ \begin{bmatrix} b_1 & b_2 \end{bmatrix}, \begin{bmatrix} b_5 & b_6 \end{bmatrix} \right\}.$$

Furthermore, we denote the flattening of a rank 3 tensor by $\text{mat} : \mathbb{R}^{b \times n \times m} \rightarrow \mathbb{R}^{bn \times m}$, which amounts to combining the first and second dimension. For example,

$$\text{mat} \left(\left\{ \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \end{bmatrix}, \begin{bmatrix} a_5 & a_6 \\ a_7 & a_8 \end{bmatrix} \right\} \right) = \begin{bmatrix} a_1 & a_2 \\ a_3 & a_4 \\ a_5 & a_6 \\ a_7 & a_8 \end{bmatrix}.$$

For a tensor composed of matrices $\mathcal{Q} = (Q_1, Q_2, \dots, Q_M)$, the construction of a block diagonal matrix from tensor $\mathcal{Q}$ is denoted by

$$\text{blk}(\mathcal{Q}) = \begin{bmatrix} Q_1 & 0 & \dots & 0 \\ 0 & Q_2 & \dots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & \dots & 0 & Q_M \end{bmatrix}.$$

Given a matrix $F \in \mathbb{R}^{m \times n}$ where $F = [f_1, f_2, \dots, f_n]$ and f_i is the i -th column of F , the vectorization operation on F is defined as

$$\text{vec}(F) = [f_1^\top, f_2^\top, \dots, f_n^\top]^\top.$$

Moreover, for the process where a matrix is created from a vector $g = [g_1, \dots, g_l] \in \mathbb{R}^{mn}$ is defined as

$$H = Q_{V \rightarrow M}(g, m, n),$$

where (m, n) denote the dimension of H , i.e., $H \in \mathbb{R}^{m \times n}$.

d: Broadcasting Hadamard Product

In order to perform the Hadamard product between tensors of different dimensions we define the broadcasting operation and the broadcasting Hadamard product. Consider two rank 3 tensors $\mathcal{D} \in \mathbb{R}^{L_1 \times L_2 \times L_3}$ and $\mathcal{E} \in \mathbb{R}^{J_1 \times J_2 \times J_3}$. Then the broadcasting operation can be used to perform the Hadamard product between tensors of different dimensions, $\mathcal{F} = \mathcal{D} \square \mathcal{E}$, as follows,

$$\begin{aligned} \mathcal{F} &= \mathcal{D} \square \mathcal{E} := \text{bc}(\mathcal{D}, \text{size}(\mathcal{E})) \odot \text{bc}(\mathcal{E}, \text{size}(\mathcal{D})), \\ \mathcal{F} &\in \mathbb{R}^{\max(L_1, J_1) \times \max(L_2, J_2) \times \max(L_3, J_3)}. \end{aligned}$$

Above $\text{size} : \mathbb{R}^{n \times m \times p} \rightarrow \mathbb{N} \times \mathbb{N} \times \mathbb{N}$ returns the shape of the input tensor as an n -tuple such that $\text{size}(\mathcal{J}) = (J_1, J_2, J_3)$. Furthermore, the broadcasting function $\text{bc} : \mathbb{R}^{L_1 \times L_2 \times L_3} \times (\mathbb{N} \times \mathbb{N} \times \mathbb{N}) \rightarrow \mathbb{R}^{K_1 \times K_2 \times K_3}$, where $K_1 \geq L_1$, $K_2 \geq L_2$ and $K_3 \geq L_3$, is a function that has a tensor and n -tuple as arguments such that for tensor $\mathcal{D}$ and $\text{size}(\mathcal{E})$ for all n , if $L_n = 1$, replicates all elements of $\mathcal{D}$ along the n -th mode J_n times such that $\mathcal{D}' = \text{bc}(\mathcal{D}, \text{size}(\mathcal{E}))$. To illustrate the application of the broadcasting Hadamard product, consider the following example for a matrix and a column vector:

$$\begin{bmatrix} f_1 & f_2 \\ f_3 & f_4 \end{bmatrix} \square \begin{bmatrix} g_1 \\ g_2 \end{bmatrix} = \begin{bmatrix} f_1 & f_2 \\ f_3 & f_4 \end{bmatrix} \odot \begin{bmatrix} g_1 & g_1 \\ g_2 & g_2 \end{bmatrix}.$$

Similarly, for a matrix and row vector:

$$\begin{bmatrix} f_1 & f_2 \\ f_3 & f_4 \end{bmatrix} \square [g_1 \ g_2] = \begin{bmatrix} f_1 & f_2 \\ f_3 & f_4 \end{bmatrix} \odot \begin{bmatrix} g_1 & g_2 \\ g_1 & g_2 \end{bmatrix}.$$

For a tensor operation consider the following example,

$$\begin{aligned} \left\{ \begin{bmatrix} a_1 \\ a_2 \end{bmatrix}, \begin{bmatrix} a_3 \\ a_4 \end{bmatrix} \right\} \square \left\{ \begin{bmatrix} a_5 & a_5 \\ a_6 & a_6 \end{bmatrix}, \begin{bmatrix} a_7 & a_7 \\ a_8 & a_8 \end{bmatrix} \right\} = \\ \left\{ \begin{bmatrix} a_1 & a_1 \\ a_2 & a_2 \end{bmatrix}, \begin{bmatrix} a_3 & a_3 \\ a_4 & a_4 \end{bmatrix} \right\} \odot \left\{ \begin{bmatrix} a_5 & a_5 \\ a_6 & a_6 \end{bmatrix}, \begin{bmatrix} a_7 & a_7 \\ a_8 & a_8 \end{bmatrix} \right\}. \end{aligned}$$

For a formal definition consider the following work [17]. To perform the broadcasted Hadamard product, a necessary condition must be satisfied for the broadcasted Hadamard product to be well-defined. The broadcasting condition for $\mathcal{D}$ and $\mathcal{E}$ holds if for any $n \in \{1, 2, 3\}$ one of the following conditions is satisfied: $L_n = J_n$, $L_n = 1$ or $J_n = 1$.

e: Einstein Summation

To perform Einstein summation, consider two tensors $\mathcal{H} \in \mathbb{R}^{P \times T \times R \times E}$ and $K \in \mathbb{R}^{P \times T \times E}$. The output $L \in \mathbb{R}^{P \times T \times R}$

is obtained by contracting the common dimension E as follows:

$$L_{p,t,r} = \sum_{e=1}^E \mathcal{H}_{p,t,r,e} K_{p,t,e},$$

where the indices p, t, r correspond to the free dimensions and e is the summation index.

III. MPC Preliminaries and Problem Statement

Consider a continuous time-invariant nonlinear MIMO state-space system, i.e.,

$$\dot{x}(t) = f_c(x(t), u(t)) \quad (1a)$$

$$y(t) = h_c(x(t), u(t)) \quad (1b)$$

with inputs $u \in \mathbb{U} \subset \mathbb{R}^{n_u}$, states $x \in \mathbb{X} \subset \mathbb{R}^{n_x}$ and outputs $y \in \mathbb{Y} \subset \mathbb{R}^{n_y}$, time $t \in \mathbb{R}_{\geq 0}$. Above $f_c : \mathbb{U} \times \mathbb{X} \rightarrow \mathbb{Y}$ and $h_c : \mathbb{U} \times \mathbb{X} \rightarrow \mathbb{Y}$ are unknown suitable functions that allow for existence of unique solutions.

To perform sequential computation on (1), the model needs to be discretized, for example, by means of zero-order hold (ZOH). In general, a discrete-time nonlinear MIMO system is described by, i.e.,

$$x(k+1) = f(x(k), u(k)), \quad (2a)$$

$$y(k) = h(x(k), u(k)), \quad (2b)$$

where for a discrete time instant $k \in \mathbb{N}$, the continuous time $t = kT_s$, where $T_s \in \mathbb{R}$ represents the sampling time/period. In discrete-time predictive control, sequences of future inputs and predicted outputs over a prediction horizon $N \in \mathbb{N}_{\geq 1}$ are generally of interest.

Consider thus a sequence of inputs $u_{[0, N-1]}(k)$ and outputs $y_{[1, N]}(k)$ described by:

$$u_{[0, N-1]}(k) := \text{col}([u(0|k), \dots, u(N-1|k)]), \quad (3a)$$

$$y_{[1, N]}(k) := \text{col}([y(1|k), \dots, y(N|k)]), \quad (3b)$$

where the input sequence is mapped to a sequence of outputs for any initial condition by a suitable map $\mathbb{F} : \mathbb{R}^{n_u N} \times \mathbb{R}^{n_x} \rightarrow \mathbb{R}^{n_y N}$. Assuming that a suitable map $\mathbb{F}$ can accurately describe the Seq2Seq multi-step representation of the MIMO

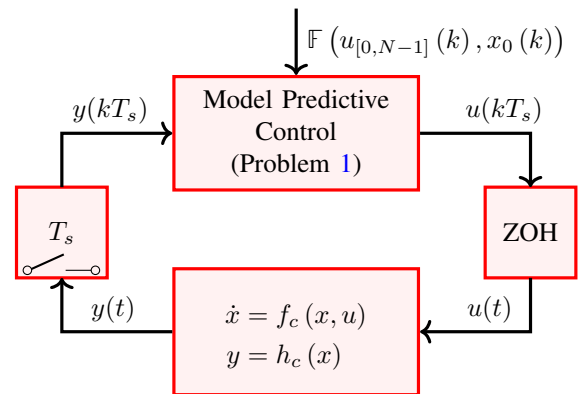


FIGURE 1: Illustration of the Model Predictive Control closed-loop control architecture.

nonlinear system in (2), then a prototype predictive control problem can be formulated as follows:

Problem 1. Prototype MPC problem

$$\min_{u_{[0,N-1]}(k), y_{[1,N]}(k)} l_N(y(N|k)) + \sum_{i=0}^{N-1} l(y(i|k), u(i|k))$$

subject to:

$$y_{[1,N]}(k) = \mathbb{F}(u_{[0,N-1]}(k), x_0(k)),$$

$$u_{[0,N-1]}(k) \in \mathbb{U} \times \dots \times \mathbb{U},$$

$$y_{[1,N]}(k) \in \mathbb{Y} \times \dots \times \mathbb{Y},$$

$$y(0|k) = y(k).$$

(4)

In (4) $x_0(k)$ is the initial condition that can be the initial state $x(k)$, if the measured states are available. $\mathbb{F}$ is a suitable map such that the sequence of inputs $u_{[0,N-1]}(k)$ and the initial condition $x_0(k)$ are mapped to the output sequence $y_{[1,N]}(k)$. $\mathbb{F}$ can be explicitly constructed if the functions f, h are known, by repeated appropriate function compositions. An illustration of using Problem 1 to define a closed-loop control architecture is provided in Figure 1.

For the learning of the map $\mathbb{F}$, several approaches can be considered. A common data-driven approach is to utilize NN architectures for the multi-step predictor. A feedforward NN was applied as multi-step predictor in MPC [2] and achieved promising results. In [10] a multi-step predictor is constructed based on a Transformer network and is successfully applied in the MPC framework but at significant computational cost. Another approach is operator networks such as DeepONet [14], which were recently adapted for implementation in MPC in [18]. In [15] a comparative study is done involving DeepONet, Transformer based approaches, and Mamba for the modeling of dynamical systems. Mamba consistently achieves superior performance with respect to DeepONet and Transformer based approaches. Furthermore, Mamba was inherently designed for linear computational scaling with respect to the prediction horizon. Consequently, Mamba is of significant interest as an application to learn nonlinear mappings for multi-step prediction in the MPC framework. In particular, Mamba-based decoder-only architectures are of interest, such that the total parameter count and therefore computational complexity in Problem 1 remains minimal. Therefore, the problem statement can be summarized as follows.

Problem statement: Given the discrete-time nonlinear MIMO system in (2), the goal is to approximate the multi-step Seq2Seq map $\mathbb{F}$ from future inputs $u_{[0,N-1]}(k)$ to future outputs $y_{[1,N]}(k)$ for a set of initial conditions $x_0(k)$ by means of Mamba NNs. The input of the network should have a sequence length equal to the sequence length of the output such that decoder-only structures can be utilized for computational tractability. The model should predict a multi-step sequence in a single forward pass, such that efficient predictive control is enabled.

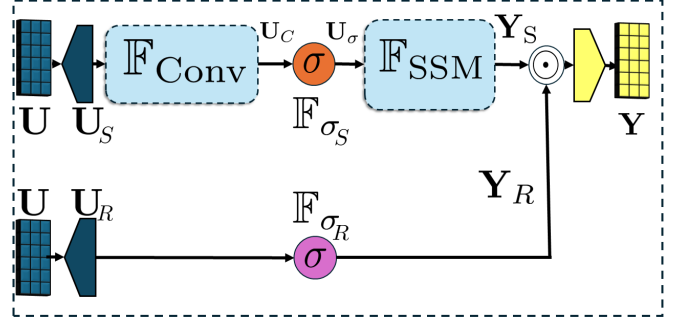


FIGURE 2: A schematic of the Mamba block architecture. The input U is processed through two parallel paths: a main path featuring a convolution layer, a hidden layer of activation functions, a SSM block that implements a time-varying linear dynamical system in the feature space, and a residual path with a hidden layer of activation functions. The final output is the element-wise product of the outputs of these two paths.

Remark 1. If state data is unavailable, a stacked vector of past inputs and outputs can be used that encode the state information. In the rest of the paper we assume that state data is available.

IV. Mamba: Mathematical Description

Let the map from input to output be described by $Y = \mathbb{F}_{\text{Mamba}}(U)$ where $\mathbb{F}_{\text{Mamba}}(U)$ is defined according to:

$$\mathbb{F}_{\text{Mamba}}(U) := (\mathbb{F}_{\text{SSM}} \circ \sigma(\mathbb{F}_{\text{Conv}}(UW_S^T)) \odot Y_R) W_Y^T \quad (5)$$

where

$$Y_R = \sigma(UW_R^T), \quad (6a)$$

$$\sigma(U_C) = U_C \odot \frac{1}{1 + \exp(-U_C)}. \quad (6b)$$

In F_{Mamba} , the functions $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ in (5) and (6a) refer to an element-wise *Sigmoid Linear Unit* (SiLU) activation function [19]. Consequently, the exponential function in (6b) is an element-wise function. Furthermore, mappings $W_S^T, W_R^T : \mathbb{R}^{L \times D} \rightarrow \mathbb{R}^{L \times ED}$ are lifting operators that expand the feature space of the input, while $W_Y^T : \mathbb{R}^{L \times ED} \rightarrow \mathbb{R}^{L \times D}$ are projection operators that reduce the feature space. The functions $\mathbb{F}_{\text{SSM}}$ and $\mathbb{F}_{\text{Conv}}$ require a more detailed description and will be further highlighted in the following sections. To illustrate the general structure of the Mamba function, a graphical description can be seen in Figure 2. Within the architecture of Mamba, an input is typically provided as a tensor denoted by $U \in \mathbb{R}^{B_a \times L \times D}$ where the parameter definitions are given in Table 1. For the following section, the dimension B_a will be considered of size one, such that $U = \text{mat}(U) \in \mathbb{R}^{L \times D}$, for simplicity.

A. 1D Convolution

Next, a causal depthwise 1 dimensional convolutional layer (1D Conv) is applied along the sequence length dimension L of the expanded input. The addition of 1D Conv in Mamba is such that linear combinations of features of prior sequence

TABLE 1: Model Parameter Definitions

Parameter	Definition
B_a	Batch Size
L	Sequence Length
D	Model Dimension
E	Expansion Factor
N	Prediction Horizon
S	Hidden State Dimension
K	Convolutional Kernel Size
d_τ	Rank of Δ Approximation

steps can be incorporated into the current sequence step. Hence, this aids in the modeling linear temporal dependencies. An illustration of the operation is given in Figure 3.

Let the 1D Conv function $\mathbb{F}_{\text{Conv}}$ be described by:

$$\mathbb{F}_{\text{Conv}}(U_S) := Q_{V \rightarrow M}(\tilde{u}, ED, L)^\top, \quad (7)$$

where

$$\tilde{u} = T \text{vec}(\tilde{U}) + \tilde{b}, \quad (8a)$$

$$T = \text{blk}(\mathcal{T}), \quad (8b)$$

$$\mathcal{T} = \text{Toeplitz}(\kappa_{d,:}), d = 1, \dots, ED, \quad (8c)$$

$$\tilde{U} = [\bar{0}_l, U_S^\top, \bar{0}_r], \quad (8d)$$

$$\tilde{b} = \text{col}(b, \dots, b) \in \mathbb{R}^{ED \cdot L}. \quad (8e)$$

To further expand on the 1D Conv operation, each step will be highlighted. First, the input is transposed $U_S^\top \in \mathbb{R}^{ED \times L}$ such that the 1D Conv is applied along the sequence length. Furthermore, the input is padded to maintain the input and output size. The padding is added to the input matrix $\tilde{U}$ such that

$$\tilde{U} = [\bar{0}_l, U_S^\top, \bar{0}_r] \in \mathbb{R}^{ED \times L + K - 1}, \quad (9)$$

where for kernel size $K \in \mathbb{N}$ if kernel size $K - 1$ is odd, then padding is applied asymmetrically such that $\bar{0}_l \in \mathbb{R}^{ED \times \lceil (K-1)/2 \rceil}$ and $\bar{0}_r \in \mathbb{R}^{ED \times \lfloor (K-1)/2 \rfloor}$. Afterwards, the input $\tilde{U}$ is flattened to a vector

$$\tilde{u} = \text{vec}(\tilde{U}) \in \mathbb{R}^{ED(L+K-1)}. \quad (10)$$

Next, because a depthwise 1D Conv is utilized, consider a learnable set of weighted sliding windows $\kappa \in \mathbb{R}^{ED \times K}$, where for every expanded model dimension $[1, \dots, ED]$ there is a unique sliding window

$$\kappa_{d,:} = [k_{d,1}, \dots, k_{d,K}], \quad (11)$$

that is reconstructed into a block diagonal matrix consisting of unique Toeplitz matrices, for every model dimension,

$$\mathcal{T} = \text{Toeplitz}(\kappa_{d,:}), d = 1, \dots, ED, \quad (12a)$$

$$T = \text{blk}(\mathcal{T}). \quad (12b)$$

Lastly, a bias term is added to each model dimension by constructing a vector by repeating each bias b , L times, such that

$$\tilde{b} = \text{col}(b, \dots, b). \quad (13)$$

In the following section, the SSM block will be explained in mathematical detail.

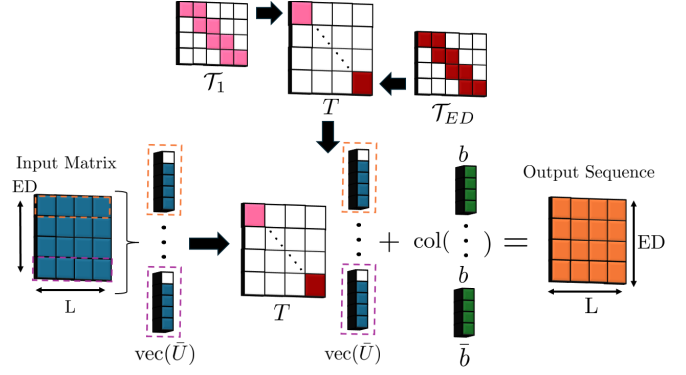


FIGURE 3: An illustration of the 1D depthwise causal convolution operation implemented as a matrix multiplication. The convolution kernel for each dimension d is transformed into a Toeplitz matrix T_d , which are then combined into a block-diagonal matrix T . This allows the entire depthwise convolution to be computed efficiently in parallel.

B. SSM

The SSM function is the core innovation of Mamba, which implements a time-varying content-aware recurrent mechanism for sequence modeling. Let $\mathbb{F}_{SSM}$ describe the SSM function:

$$Y_S = \mathbb{F}_{SSM}(U_\sigma) := H_L C + U_\sigma \square D, \quad (14)$$

where the input components are constructed by two sets of operations, i.e., first, the state-space model

$$H_\tau = \bar{A}_\tau \odot H_{\tau-1} + \bar{B}_\tau \square U_{\sigma,\tau,:}, \quad (15)$$

and second, the discretization operations

$$\bar{A} = \exp(\Delta_\tau \square A), \quad (16a)$$

$$\bar{B} = \Delta_\tau \square U_\sigma W_B^\top, \quad (16b)$$

$$\Delta_\tau = F_{NN}(U_\sigma W_\Delta^\top), \quad (16c)$$

$$C = U_\sigma W_C^\top. \quad (16d)$$

To elaborate on the SSM function, the state-space modeling and discretization steps are further clarified.

First, the discretization algorithm is considered, which incorporates a context aware selection mechanism. This is achieved by constructing $B, C \in \mathbb{R}^{L \times S}$ and $\Delta \in \mathbb{R}^{L \times d_t}$ from the input U_σ by means of maps $W_B^\top : \mathbb{R}^{L \times ED} \rightarrow \mathbb{R}^{L \times S}$, $W_C^\top : \mathbb{R}^{L \times ED} \rightarrow \mathbb{R}^{L \times S}$ and $W_\Delta^\top : \mathbb{R}^{L \times ED} \rightarrow \mathbb{R}^{L \times d_t}$:

$$B = U_\sigma W_B^\top, \quad (17a)$$

$$C = U_\sigma W_C^\top, \quad (17b)$$

$$\Delta = U_\sigma W_\Delta^\top. \quad (17c)$$

By making B and C input dependent, B becomes analogous to an input selection gate and C becomes the output selection gate. To finalize the construction of the sampling time matrix, a softplus activation is applied, element-wise, to allow Δ_t to be a nonlinear function that can describe time warping [13]. Furthermore, a map $W_{\Delta_\tau}^\top : \mathbb{R}^{L \times d_t} \rightarrow \mathbb{R}^{L \times ED}$ and bias term b_{Δ_τ} are added. Thus, Δ_τ is described by:

$$\Delta_\tau = \ln(1 + \exp(\Delta W_{\Delta_\tau}^\top + b_{\Delta_\tau})) = F_{NN}(U_\sigma), \quad (18)$$

which can be seen as a feedforward NN. The application of a softplus activation function is motivated by the connection to the GRU gating mechanism, which is shown in [11], where, similarly to B and C , by constructing Δ_τ from the input, Δ_τ becomes analogous to a gate that dynamically focuses on past sequences or current inputs. The temporal focus is achieved by applying discretizations to parameters $A \in \mathbb{R}^{ED \times S}$ and B to construct $\bar{A}$ and $\bar{B}$ such that $\bar{A}$ dictates what information in hidden states H_τ should be propagated versus what input information should be stored in H_τ through $\bar{B}$. A graphical illustration is given in Figure 4.

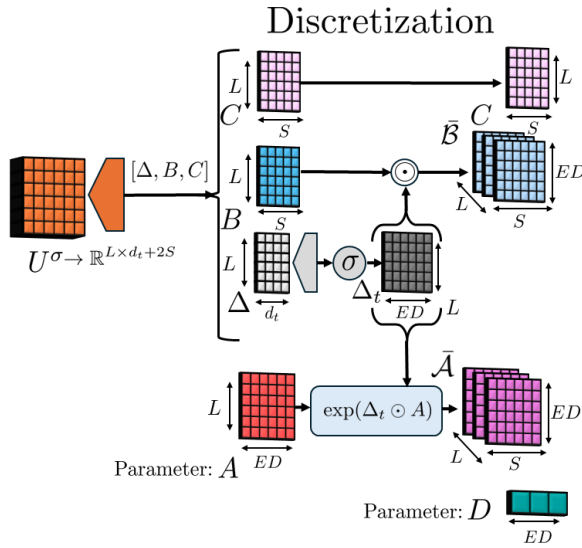


FIGURE 4: A visual representation of the discretization algorithm in the SSM function.

Remark 2. It should be noted that A is a dense matrix where the row vectors contain the diagonal elements of individual matrices. Therefore, tensor $\bar{A}(U_\sigma)$ is a dense tensor where for every sequence step $l = 1, \dots, L$ there is a unique matrix $\bar{A}_l$. Furthermore, every column vector in matrix $\bar{A}_l \in \mathbb{R}^{ED \times S}$ contains the diagonal elements of matrix $\bar{A}_{l,d}$ such that for every sequence step in L , there exist ED parallel SSM, as can be seen in Figure 5. The fact that A is a diagonal matrix is a key component because it allows the ZOH to be approximated using the exponents of the diagonal components rather than a matrix exponential, which would lead to unstable gradients [20].

Next we present the state-space modeling algorithm, which is fundamentally based on (2). For Mamba, a parallelized description of (2) is used which implements the context aware matrices of (16). Furthermore, a trainable feed-through parameter $D \in \mathbb{R}^{ED}$ is introduced. Mamba's

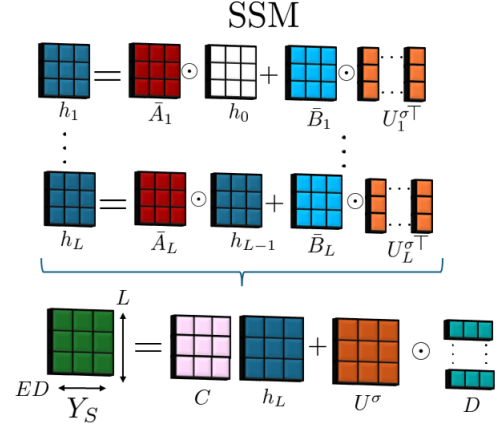


FIGURE 5: A visual representation of the selective state-space block in a Mamba NN

parallelized discrete SSM is described by:

$$H_\tau = \bar{A}_\tau \odot H_{\tau-1} + \bar{B}_\tau \boxtimes U_{\sigma\tau,:}, \quad (19)$$

where $\tau = 1, \dots, L$ is fictitious time, H_0 is initialized as a zero matrix, and H_τ can be partitioned as $H_\tau = [h_{\tau,1}, \dots, h_{\tau,ED}] \in \mathbb{R}^{ED \times S}$. The output of the SSM layer is constructed by:

$$Y_S = \sum_{s=1}^S H_{L,b,l,d,s} C_{b,l,s} + U_\sigma \boxtimes D. \quad (20)$$

An illustration of how the SSM mechanism operates is given in Figure 5. This can be regarded as an L step-ahead time-varying linear predictor in the feature space.

Remark 3. We emphasize that the parallelized description given in (19) is equivalent to a time-varying linear state-space model.

Now that a consistent and complete mathematical description is provided for Mamba, a graphical illustration of the full model is given in Figure 6.

In the next section, we will discuss the adaptation required for implementation of Mamba as a multi-step predictor in Model Predictive Control.

V. Mamba for Predictive Control

In this section, a novel implementation of Mamba for predictive control is introduced that allows Mamba to learn the Seq2Seq mapping for an arbitrary initial state. To this end, a novel embedding is introduced of $u_{[0,N-1]}(k)$ and $x_0(k)$ that maintains the same input sequence length as output sequence length for $y_{[1,N]}(k)$ such that the usage of a decoder-only architecture is enabled.

In the previous Section IV, L was used to describe the length of a sequence, but in control systems it is common to use prediction horizon N . From this section onward, the prediction horizon N will be used instead of the sequence length L .

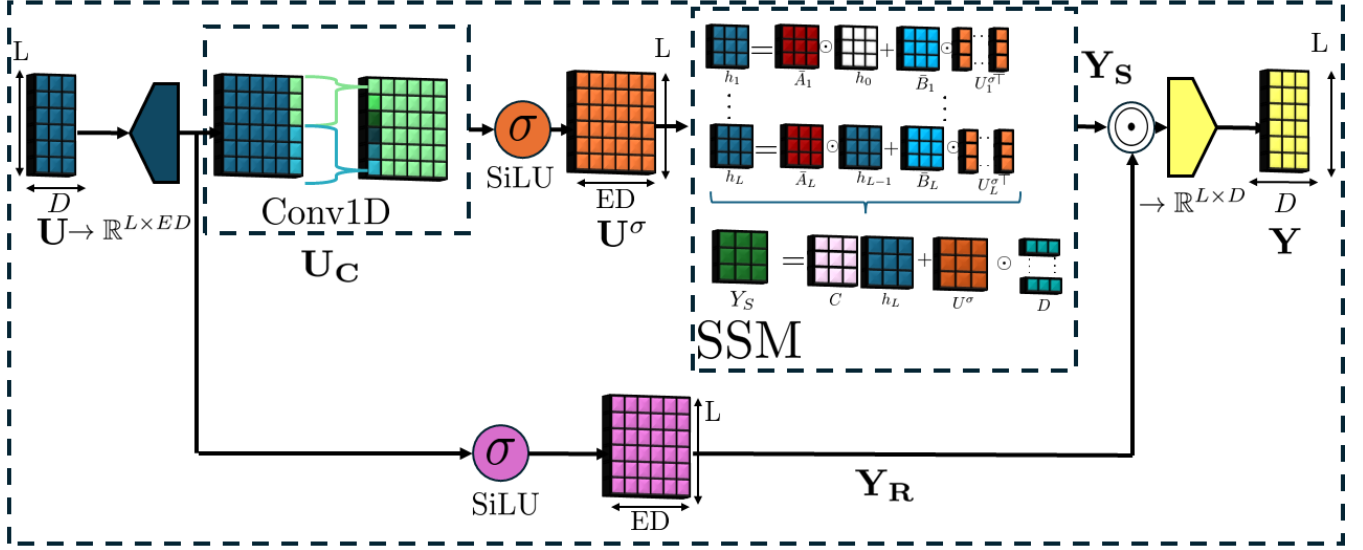


FIGURE 6: A complete schematic of the Mamba block-wise architecture (for simplicity of illustration, the two different input lifting operators corresponding to the two main parallel paths are merged into one).

A. Data Embedding

In a decoder-only architecture utilizing Mamba, no changes are made to the sequence length dimension. Therefore, an alternative input embedding can be constructed where the input sequence is concatenated with initial condition columns.

Consider a dataset $\mathcal{D} =: [X_0, U_f, Y_f]$ with amount of samples T

$$X_0 = [x_0(0), \dots, x_0(T-1)], \quad (21a)$$

$$U_f = [u_{[0, N-1]}(0)^\top, \dots, u_{[0, N-1]}(T-1)^\top], \quad (21b)$$

$$Y_f = [y_{[1, N]}(1)^\top, \dots, y_{[1, N]}(T)^\top], \quad (21c)$$

where $X_0 \in \mathbb{R}^{T \times 1 \times n_x}$, $U_f \in \mathbb{R}^{T \times N \times n_u}$ and $Y_f \in \mathbb{R}^{T \times N \times n_y}$. Then the following input embedding can be constructed for each sample in the dataset.

Definition 1. *Input and output embedding for MPC*

$$U_{IC}(k) = \mathbb{F}_{IC}(u_{[0, N-1]}(k), x_0(k)) := \begin{bmatrix} u(0|k)^\top & x(0|k)^\top \\ u(1|k)^\top & x(0|k)^\top \\ \vdots & \vdots \\ u(N-1|k)^\top & x(0|k)^\top \end{bmatrix} \in \mathbb{R}^{N \times n_u + n_x} \quad (22)$$

where $U_{IC}(k)$ is constructed for each sample in dataset $\mathcal{D}$. Furthermore, the output embedding for every sample in the batch is given by;

$$Y(k) := \begin{bmatrix} y(1|k)^\top \\ \vdots \\ y(N|k)^\top \end{bmatrix} \in \mathbb{R}^{N \times n_y}. \quad (23)$$

The structure given in (22) allows the embedding of an arbitrary number of initial states, inputs, and outputs and maintains the sequence length.

B. Mamba-MPC

To effectively construct a Seq2Seq mapping from embedding (22) while utilizing Mamba, adjustments are incorporated. Let $\mathbb{F}_{MPC}(U_{IC})$ denote the function describing the Mamba-MPC function, where $\mathbb{F}_{MPC}(U_{IC})$ is described by:

$$\hat{Y} = \mathbb{F}_{MPC}(U_{IC}) := \mathbb{F}_{\hat{Y}} \circ \mathbb{F}_{RMS}(\mathbb{F}_{Mamba}(U) + U_{in}, W_{RMS_2}) \quad (24)$$

where

$$U_{in} = \mathbb{F}_E(U_{IC}) \quad (25a)$$

$$U = \mathbb{F}_{RMS}(U_{in}, W_{RMS_1}). \quad (25b)$$

Although (24) conceptually contains a single $\mathbb{F}_{Mamba}$, multiple Mamba layers can be cascaded when required. As introduced in (5), the input to Mamba must be of dimension D . Therefore, the input embedding is projected to the input dimension D by the map $W_E : \mathbb{R}^{B \times N \times n_u + n_x} \rightarrow \mathbb{R}^{B \times N \times D}$ such that:

$$U_{in} = \mathbb{F}_E(U_{IC}) := U_{IC} W_E^\top + b_E, \quad (26)$$

where b_E is a bias term.

Furthermore, the input to the Mamba block needs to be normalized to allow deeper networks and gradient conditioning [21]. The choice was made for Root-Mean-Square-Layer-Normalization (RMS norm) [22] due to the lower computational cost compared to layer normalization. In (24) there are two RMS norm layers present, which, conceptually, are the same. However, each layer has unique parameters. Generally, the RMS function can be described as:

$$\mathbb{F}_{RMS}(U, W_{RMS}) := W_{RMS} \odot \frac{U}{\sqrt{\frac{1}{D} \sum_{k=1}^D (U_{:,k})^2 + \epsilon}} \quad (27)$$

where W_{RMS} is the weighting, which is a learnable parameter, and ϵ is a small static bias. Furthermore, a residual

connection from the output of $\mathbb{F}_E$ is added to improve gradient flow. Therefore, the output of the second RMS norm is given by:

$$Y_{\text{RMS}} = \mathbb{F}_{\text{RMS}}(Y_{\text{Mamba}} + U_{\text{in}}, W_{\text{RMS}_2}) \quad (28)$$

Remark 4. *Although residual connections are commonly used for gradient flow, residual connections also add linear time invariant (LTI) components to the output which simplifies the learning problem. Therefore, Mamba-MPC can be separated into an LTI component, a nonlinear time invariant component in the Mamba residual branch (6a), and a temporal modeling branch in the SSM block (14).*

Lastly, a linear projection is utilized to map the model dimension to the output dimension by means of map $W_{\hat{Y}} : \mathbb{R}^{B \times N \times D} \rightarrow \mathbb{R}^{B \times N \times n_y}$:

$$\hat{Y} = Y_{\text{RMS}} W_{\hat{Y}}^\top + b_{\hat{Y}}, \quad (29)$$

where $b_{\hat{Y}}$ is a bias term.

Now that the network structure of Mamba-MPC has been defined, a model learning problem can be constructed:

Problem 2. *Mamba-MPC learning problem*

$$\arg \min J(Y^{\text{train}}, \hat{Y}^{\text{train}}) = \frac{\|Y^{\text{train}} - \hat{Y}^{\text{train}}\|_2^2}{\|Y^{\text{train}}\|_2^2} \quad (30a)$$

subject to:

$$\hat{Y}^{\text{train}} = \mathbb{F}_{\text{MPC}} \circ \mathbb{F}_{\text{IC}}(U_f^{\text{train}}, X_0^{\text{train}}). \quad (30b)$$

Above U_f^{train} is the set of input sequences, X_0^{train} is the set of initial conditions and Y^{train} is the set output sequences, when used in training. In conjunction, checkpointing is applied to avoid overfitting and ensure sufficient exploration of the loss landscape.

Next, the prediction model can be used in the predictive control for tracking a reference $r(k) = [r(0|k), \dots, r(N|k)]$, with the error between the output sequence and the reference defined as $\varepsilon_{y[0,N]} := y_{[0,N]}(k) - r_{[0,N]}(k)$. Furthermore, the change in input is penalized given by $\Delta u_{[0,N-1]}(k) = u_{[0,N-1]}(k) - u_{[-1,N-2]}(k)$, where $u(-1|k) := u(k-1)$.

Problem 3. *Mamba-MPC*

$$\min_{\substack{\varepsilon_{y[0,N]}^\top \Omega \varepsilon_{y[0,N]} + \Delta u_{[0,N-1]}(k)^\top \Psi \Delta u_{[0,N-1]}(k) \\ y_{[0,N-1]}(k) \\ y_{[1,N]}(k)}}$$

subject to:

$$y_{[1,N]}(k) = \mathbb{F}_{\text{MPC}} \circ \mathbb{F}_{\text{IC}}(u_{[0,N-1]}(k), x_0(k)),$$

$$u_{[0,N-1]}(k) \in \mathbb{U} \times \dots \times \mathbb{U},$$

$$y_{[1,N]}(k) \in \mathbb{Y} \times \dots \times \mathbb{Y},$$

$$y(0|k) = y(k).$$

Above the quadratic cost function weight matrices are defined as follows:

$$\Omega = \begin{bmatrix} Q & & & \\ & \ddots & & \\ & & Q & \\ & & & P \end{bmatrix}, \quad \Psi = \begin{bmatrix} R & & & \\ & \ddots & & \\ & & & R \end{bmatrix}.$$

Given that $\mathbb{F}_{\text{MPC}}$ is constructed by Mamba, this problem is considered non-convex while being locally Lipschitz and globally differentiable. Therefore, non-linear optimizers such as IPOPT and SQP solvers will be considered. In the next section, we analyze the performance of Mamba-MPC as defined in Problem 3 by means of 2 nonlinear benchmark examples.

C. Towards Stability Guarantees for Mamba-MPC

Proving stability of Mamba-MPC is non-trivial due to the complex heterogeneous Mamba architecture, which includes convolution operations, time-varying discretization, recurrent linear models and Hadamard products of feedforward neural network outputs. A promising framework for addressing stability of MPC with general input-output predictors, such as neural networks, was recently developed in [23]. Therein, exponential stability is established without terminal ingredients under the assumptions that both the prediction model and the system are Lipschitz continuous, and that the predictor satisfies proportional error bounds. These results could be applied to Mamba-MPC provided that Lipschitz bounds can be established for the Mamba NN architecture.

While a complete stability analysis is beyond the scope of this paper, in what follows we point out how Lipschitz bounds can be derived for the Mamba network. To this end we first introduce the definition of Lipschitz continuity tailored to our needs.

Definition 2. *A map $F : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}^p$ is Lipschitz continuous w.r.t. its first argument uniformly in its second argument if there exists a Lipschitz constant $L > 0$, such that*

$$\|F(x, u) - F(x', u)\| \leq L\|x - x'\|, \quad \forall u \in \mathbb{U}$$

for all $x, x' \in \mathbb{X}$.

Next, establishing Lipschitz continuity of the Mamba network amounts to verifying that each of its underlying functions/maps is Lipschitz continuous. In particular, the network employs the SiLU activation function (6b) and the Softplus activation function (18), both of which are known to be globally Lipschitz (see, e.g., [24], [25]). Consequently, these nonlinearities do not obstruct the Lipschitz continuity of the overall network.

Another operation performed in the network is the discretization of the state-space equation (16a), implemented in the SSM block and involving a matrix exponential term. This exponential is applied element-wise. It is well known that exponential functions are not globally Lipschitz on $\mathbb{R}$. However, they are locally Lipschitz, which can be shown directly. Let $f(x) = e^x$, and consider any bounded interval $[a, b] \subset \mathbb{R}$. For any $x, y \in [a, b]$, the Mean Value Theorem guarantees the existence of some c between x and y such that $e^x - e^y = e^c(x - y)$. Taking absolute values yields

$$|e^x - e^y| = e^c|x - y| \leq e^b|x - y|$$

where $b \geq c$. Thus, e^x is Lipschitz continuous on every bounded interval, with Lipschitz constant e^b . Consequently, e^x is locally Lipschitz on $\mathbb{R}$.

Another concern that might break the Lipschitz continuity of the Mamba network is the case when the state-space model (19) is unstable, and states might grow unbounded. Since the state space model is a time varying system, stability analysis is not straightforward. This time varying behavior is introduced by the input dependent discretization (16). However, recently it was shown in [26] that the SSM recursion (19) is BIBO stable under the mild assumption that the matrix A in (16a) is Hurwitz.

Finally, the outputs of the SSM block Y_S and the parallel path Y_R are combined via element-wise multiplication, i.e., the Hadamard product. Even if both mappings are individually Lipschitz, this alone does not guarantee that their Hadamard product is Lipschitz. Additional assumptions, such as boundedness of the components, are required to ensure Lipschitz continuity of the combined map.

Lemma 1. *Let $f, g : \mathbb{R} \rightarrow \mathbb{R}$ be Lipschitz and bounded, i.e., there exist constants $L_f, L_g > 0$ such that:*

$$|f(x) - f(y)| \leq L_f |x - y|, \quad |g(x) - g(y)| \leq L_g |x - y|.$$

Then $h(x) = f(x)g(x)$ is also Lipschitz.

Proof:

Let $h(x) = f(x)g(x)$. Then

$$h(x) - h(y) = f(x)(g(x) - g(y)) + g(y)(f(x) - f(y)).$$

Taking absolute values and applying the triangle inequality gives

$$|h(x) - h(y)| \leq |f(x)| |g(x) - g(y)| + |g(y)| |f(x) - f(y)|.$$

Using the Lipschitz bounds and boundedness of $f(x) \leq M_f$ and $g(x) \leq M_g$ for all x , we obtain

$$|h(x) - h(y)| \leq (M_f L_g + M_g L_f) |x - y|.$$

Hence h is Lipschitz. $\blacksquare$

The above analysis provides the key steps for establishing local Lipschitz boundedness of the complete Mamba network architecture. This provides a useful basis for future work that will deal with a rigorous stability analysis of Mamba-MPC, for instance within the framework of [23].

VI. Validation in Numerical Examples

This section details the comprehensive evaluation of the proposed Mamba-MPC scheme across a series of performance metrics.

For all considered systems, we evaluate the performance of Mamba-MPC against a state-space LSTM [27] based MPC controller as in [18]. For the Van der Pol oscillator an additional comparison with a Transformer-MPC model was made. The Transformer-MPC model was constructed according to [10] with the addition of a temperature parameter in the self-attention mechanism. The addition of a temperature parameter was done to aid numerical stability [28]. The

utilized parameters can be found in Table 2 and Table 4 for the Van der Pol and Four tank, respectively.

TABLE 2: Comparison of Mamba-MPC and LSTM-MPC for the Van der Pol System.

	Van der Pol			
	Reference Tracking		Noise	
	Mamba	LSTM	Mamba	LSTM
<i>Model Architecture</i>				
Model Dim. D	8	2	8	2
Kernel Size	10	-	10	-
State Dim.	8	26	4	26
Horizon N	10	10	10	10
Layers	6	-	6	-
Params	3418	3052	3418	3052
<i>Controller Performance</i>				
MAE	0.066	0.072	0.07	0.09
MSE	0.058	0.066	0.03	0.04
Comp. Time (s)	0.018	0.20	0.07	0.71
Sampling Time (s)	0.1	0.1	0.1	0.1
<i>Training Metrics</i>				
Time (min)	400	35	400	35
Train Loss	$2 \cdot 10^{-5}$	$8.4 \cdot 10^{-7}$	$6.8 \cdot 10^{-5}$	$6.2 \cdot 10^{-4}$
Val. Loss	$5.5 \cdot 10^{-5}$	$6.9 \cdot 10^{-5}$	$8.8 \cdot 10^{-5}$	$2.2 \cdot 10^{-2}$

TABLE 3: Transformer-MPC.

	Van der Pol
	Reference Tracking
	Transformer
<i>Model Architecture</i>	
Model Dim. D	8
Heads	4
MLP Dim.	16
Horizon N	10
Layers	3
Params	26600
<i>Controller Performance</i>	
MAE	0.13
MSE	0.15
Comp. Time (s)	0.09
Sampling Time (s)	0.1
<i>Training Metrics</i>	
Time (min)	1200
Train Loss	$7 \cdot 10^{-5}$
Val. Loss	$2.3 \cdot 10^{-4}$

First, the SISO Van der Pol oscillator is considered where stabilization is tested for multiple initial conditions. Subsequently, a tracking performance comparison against LSTM-MPC and Transformer-MPC is made where a fair assessment is ensured by using identical datasets, MPC cost function, prediction horizon and each network is optimized for their respective optimal compromise between accuracy, size and numerical accuracy. Thereafter, a comparison is made for computational scaling against prediction horizon. Then, robustness against noise is tested during reference

tracking and compared against LSTM-MPC. Afterwards, the MIMO Four Tank benchmark is introduced where reference tracking for multiple outputs is tested and compared with NMPC, where again, a fair comparison is made in terms of MPC cost and prediction horizon. Finally, the Mamba-MPC is applied to a physical benchmark system, the Quanser Aero2, to validate real-time reference tracking performance. All results presented in this section were generated using PyTorch and CasADI frameworks which were executed on an Nvidia RTX 4070 and AMD Ryzen 9 7845HX CPU, respectively.

A. Van der Pol Oscillator

Mamba-MPC is first evaluated on the SISO Van der Pol system with control input, i.e.,

$$\dot{x}_1 = x_2, \quad (31a)$$

$$\dot{x}_2 = \mu(1 - x_1^2)x_2 + u, \quad (31b)$$

$$y = x_1, \quad (31c)$$

with $\mu = 1$, input $u \in \mathbb{R}$, output $y \in \mathbb{R}$ and initial states $x_0 \in \mathbb{R}^2$. Furthermore, the Van der Pol is discretized with forward Euler with sampling time $T_s = 0.1$ s. The specific selection of $\mu = 1$ ensures that Van der Pol exhibits limit cycle behavior, thereby constituting a non-trivial benchmark problem. A dataset consisting of $T = 40000$ samples is created via an open-loop identification experiment, in which a multisine input signal is used featuring a peak amplitude of 15, across a frequency range of approximately 0.0049 Hz to 4.88 Hz, containing 30 harmonics. A training and validation dataset is created according to (22) and utilized in learning Problem (30). A model is constructed according to the model parameters given in Table 2. The model is trained for 4000 epochs in PyTorch, using the Adam optimizer [29] with an initial learning rate $l_r = 1 \cdot 10^{-3}$, an L2 regularization cost of $\lambda = 10^{-5}$, weight decay factor of $w_t = 1 \cdot 10^{-5}$, and exponential learning rate scheduler with $\gamma = 0.998$ per 10 epochs.

The PyTorch model is implemented as a CasADI function to formulate the prediction model in 3 to enable closed-loop control. We first assess the system's ability to stabilize a system for a variety of initial conditions. To generate a diverse set of initial conditions, $x_0 = \text{col}(x_1(0), x_2(0))$, 100 realizations are drawn according to $x_1(0) \sim \text{Uniform}(-2.5, 2.5)$ and $x_2(0) \sim \text{Uniform}(-2.0, 2.0)$. For the Mamba-MPC scheme (3) the prediction horizon is $N = 10$, $Q = 50$, $R = 0.5$ and $P = 100$. This optimization problem is solved using IPOPT solver [30], which is employed for all Van der Pol examples. Consider the stabilizing closed-loop response of Mamba-MPC for the Van der Pol for 100 initial condition realizations in Figure 7. The results confirm that Mamba-MPC successfully stabilizes the system for all sampled trajectories, though one specific trajectory exhibited a higher amplitude of oscillations during the convergence phase.

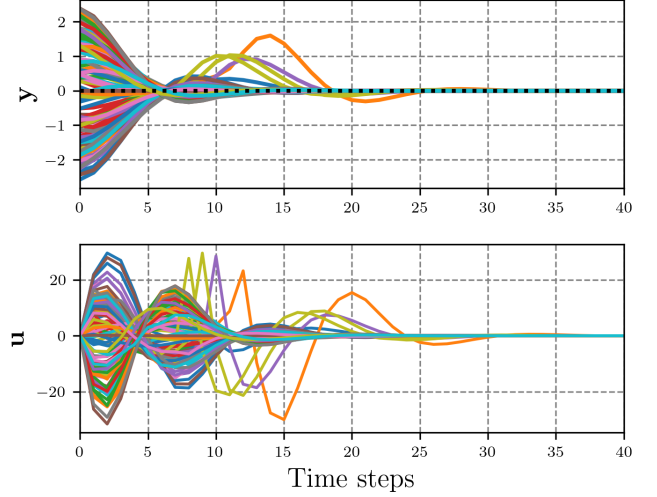


FIGURE 7: Closed-loop performance of Mamba-MPC for stabilization of a Van der Pol oscillator for 100 different initial conditions.

To evaluate the tracking performance of Mamba-MPC, a comparative analysis is conducted against a Transformer and LSTM model. Mamba-MPC, Transformer, and LSTM are trained on an identical dataset and are trained according to learning Problem (30). The choice of the LSTM as baselines is motivated by the architectural similarities between Mamba and LSTM specifically regarding their recurrent states and gating mechanism. The Transformer is chosen due to having comparable to superior modeling performance in comparison to Mamba in a wide range of modalities. As previously mentioned, a temperature parameter was added to the self-attention of the Transformer and was empirically chosen to be 2.5. The Transformer settings can be found in Table 3. To ensure an equitable comparison, the hyperparameter setting is chosen to be optimal for its architecture in terms of online computational time, modeling performance, and online numerical stability. All control schemes are implemented with a prediction horizon $N = 10$ and $Q = 100$, $R = 0.5$.

Consider the closed-loop response for tracking a piecewise constant reference signal in Figure 8. Generally, the LSTM and Mamba based MPC controllers demonstrate a comparable ability to track the reference signal, a finding substantiated when considering the Mean-Absolute-Error (MAE) and Mean-Squared-Error (MSE) in Table 2 and table 3. However, the Transformer-MPC shows significantly worse performance, particularly in regions with aggressive reference changes. In general to obtain a result, the Transformer had to be trained for $N = 50$ to generalize beyond a reference level of 2.5. We hypothesize that this is due to the numerical difficulties as well as poor generalization and requires a dedicated analysis to form a conclusion. However, when considering the mean computation time, Mamba-MPC outperforms both competitors with a computation time of 0.018s, whereas LSTM-MPC does not stay within the

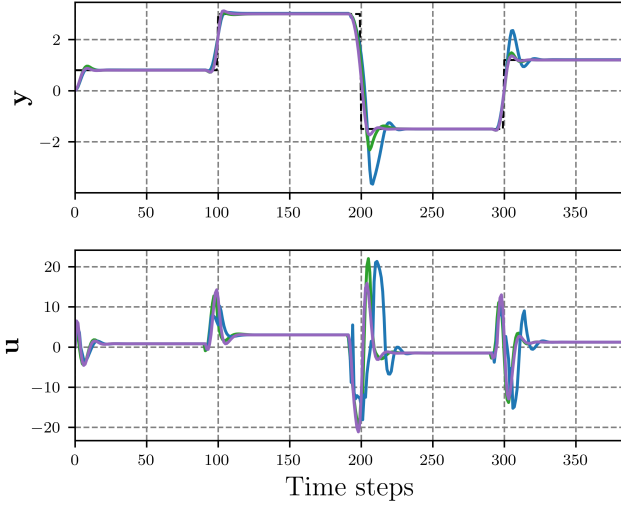


FIGURE 8: Comparison between Mamba-MPC (—), LSTM-MPC (—), and Transformer-MPC (—) for a piece-wise constant reference (· ·) on a Van der Pol oscillator.

sampling time of the system at 0.2s, and the Transformer marginally stays within the sampling time at 0.09s. To further analyze the computational scaling of each architecture, a comparison was made for a set of prediction horizons $N = [5, 10, \dots, 50]$. It is evident from Figure 9 that the computational scaling of each model is clearly visible where Mamba and LSTM scale linearly with prediction horizon while the Transformer scales quadratically. Therefore, for following examples, we omit the Transformer since it would be an infeasible approach.

Remark 5. In section V, a choice was made to reset the hidden state at each time step. This was done such that a consistent comparison of Mamba as input-output operator could be constructed. However, the question whether this is optimal remains open. To construct a brief empirical analysis between stateful and stateless training, a comparison was conducted. Identical settings as provided in Table 2 we utilized for the stateless and stateful model. To train the stateful model, an adaptation to the training and test set were done to align subsequent sequences along the batch dimension. Thus, the model can propagate $h(0)$ for each batch and maintain chronological order. The training was done for a noiseless data set. In Figure 10 the comparison can be seen. It can be noted that the performance is comparable with small deviations. However, to form a strong conclusion a broader study is required.

Lastly, we evaluate the reference tracking capabilities of Mamba-MPC under stochastic noise conditions. We perform a direct comparison with LSTM-MPC. The dataset acquired in the previous example is corrupted with a Gaussian, zero-mean, additive white noise realization such that the measured states x_0 and output y have a signal-to-noise ratio of $SNR =$

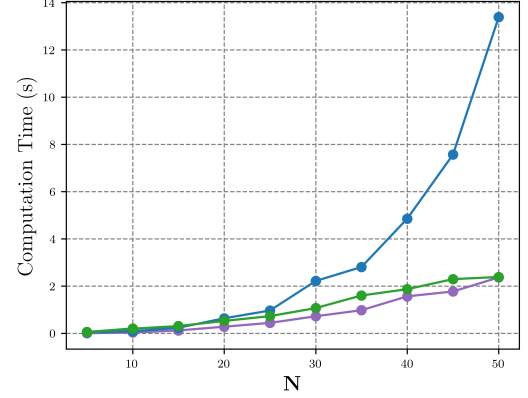


FIGURE 9: Comparison between the mean online computation time of Mamba-MPC (—), LSTM-MPC (—), and Transformer-MPC (—) on a Van der Pol oscillator.

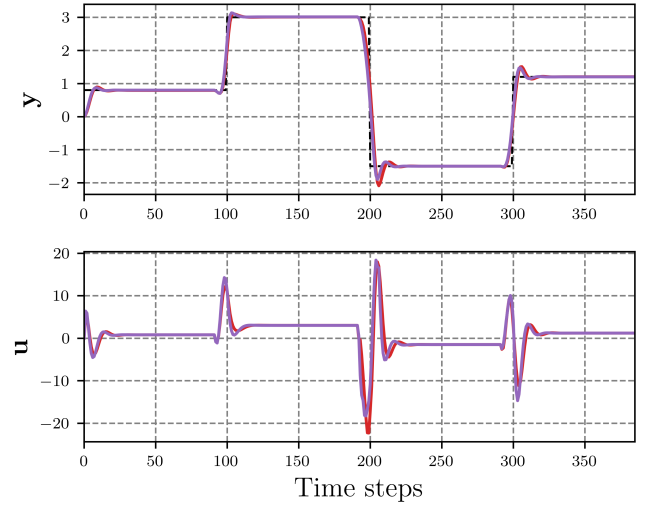


FIGURE 10: Comparison between stateless Mamba-MPC (—) and stateful Mamba-MPC (—) for a piece-wise constant reference on the Van der Pol in noiseless conditions.

20. Subsequently, both the Mamba and LSTM models are retrained according to (30) with the same settings as in the previous example.

In closed-loop control implementation, zero-mean Gaussian white noise with standard deviation $s = [0.16, 0.13]$ is added to each measured state, respectively. A total of 100 independent noise realizations are created to perform a Monte-Carlo evaluation. Both control schemes are implemented with a prediction horizon $N = 10$ and quadratic cost $Q = 50$, $R = 1$ and terminal cost $P = 10$. We quantitatively compare the control schemes for the following criteria:

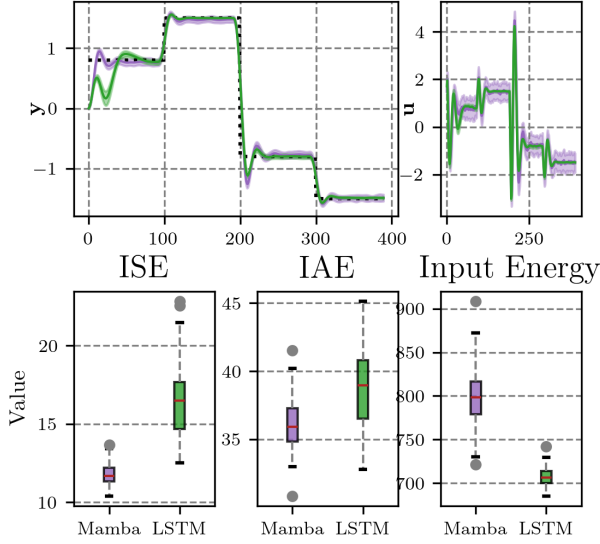


FIGURE 11: A closed-loop reference ($\bullet \cdot$) tracking comparison between Mamba-MPC (---) and LSTM-MPC (---) under noise conditions with a Signal-to-Noise Ratio (SNR) conditions of 20. The solid line indicates the mean value, and the shaded area represents the standard deviation obtained across 100 noise realizations.

$$\text{ISE} = \sum_{k=0}^{t_{\max}} \|y(k) - r(k)\|_2^2, \quad \text{IAE} = \sum_{k=0}^{t_{\max}} \|y(k) - r(k)\|_1, \\ \text{Input Energy} = \sum_{k=0}^{t_{\max}} \|u(k)\|_2^2, \quad (32)$$

where ISE indicates the integral squared error and IAE indicates the integral absolute error. The resulting mean tracking performance and associated standard deviations are presented in Figure 11 indicating that Mamba-MPC effectively mitigates measurement noise effects. Quantitatively, the ISE and IAE suggest that Mamba-MPC achieves superior tracking performance compared to LSTM-MPC, albeit demanding a higher magnitude of input energy. However, closer inspection of the tracking performance, reveals that Mamba-MPC generally exhibits a higher standard deviation than LSTM-MPC. Furthermore, LSTM-MPC maintains a lower tracking error across the simulation, excluding the start of simulation. This initial suboptimality in LSTM-MPC is likely due to its reliance on the hidden state for predictions, where the initialization of the hidden state at initial time $t = 0$ is non-trivial. Once the hidden state converges, LSTM-MPC demonstrates a higher degree of noise robustness, in comparison to Mamba-MPC. We suspect that this is due to reinitialization of the hidden states of Mamba-MPC as 0 at every control interval. Consequently, this renders Mamba-MPC more sensitive to perturbations

TABLE 4: Comparison of Mamba-MPC and LSTM-MPC for the MIMO Four Tank System.

	Four Tank (MIMO)	
	Mamba	LSTM
<i>Model Architecture</i>		
Model Dim. D	6	6
Kernel Size	20	-
State Dim.	4	20
Horizon N	20	20
Layers	1	-
Params	708	2324
<i>Controller Performance</i>		
MAE	$[2, 1, 1, 1] \cdot 10^{-2}$	$[4, 4, 5, 5] \cdot 10^{-2}$
MSE	$[4, 3, 1, 1] \cdot 10^{-3}$	$[4, 5, 5, 7] \cdot 10^{-3}$
Comp. Time (s)	0.14	0.65
Sampling Time (s)	5.0	5.0
<i>Training Metrics</i>		
Time (min)	400	180
Train Loss	$5.1 \cdot 10^{-7}$	$5.1 \cdot 10^{-5}$
Val. Loss	$1.1 \cdot 10^{-5}$	$3.2 \cdot 10^{-4}$

in the initial condition. Nevertheless, when considering the mean computation time in 2, Mamba-MPC is significantly faster than LSTM-MPC with a mean computation time of 0.07s and 0.71s, respectively. This indicates that in the presence of noise, Mamba-MPC is a reliable candidate.

B. Four Tank System

As a second example, a system with four interconnected tanks (Four Tank) is considered from [31] as a benchmark for MIMO systems, i.e.,

$$\dot{x}_1 = \frac{-a_1}{S_c} \sqrt{2gx_1} + \frac{a_3}{S_c} \sqrt{2gx_3} + \frac{\gamma_a}{3600S_c} u_1, \quad (33a)$$

$$\dot{x}_2 = \frac{-a_2}{S_c} \sqrt{2gx_2} + \frac{a_4}{S_c} \sqrt{2gx_4} + \frac{\gamma_b}{3600S_c} u_2, \quad (33b)$$

$$\dot{x}_3 = \frac{-a_3}{S_c} \sqrt{2gx_3} + \frac{(1 - \gamma_b)}{3600S_c} u_2, \quad (33c)$$

$$\dot{x}_4 = \frac{-a_4}{S_c} \sqrt{2gx_4} + \frac{(1 - \gamma_a)}{3600S_c} u_1, \quad (33d)$$

where each state $\text{col}(x_1, x_2, x_3, x_4)$ represents the water level of each of the four tanks, respectively. The parameters of the four tank system are given in Table 5. For the Four Tank example we assume that all states are measurable. Note that the system outputs and initial conditions are the states $x, x_0 \in \mathbb{R}^4$ and the inputs are $u \in \mathbb{R}^2$, hence a MIMO system. To gather data, an open-loop experiment is performed using a piece-wise-constant input signal with sampling time $T_s = 5$ and $T = 80000$ samples of input-state data are collected. The dataset is generated according to (22) and used in training according to (30). The model is trained for a total of 3000 epochs with the Adam optimizer [29] with identical hyperparameters and settings as in the Van der Pol example. A model is trained with parameters given in 4.

Similarly to the Van der Pol example, an evaluation of Mamba-MPC is made against LSTM-MPC. The models are

TABLE 5: Parameters for the Four-Tank System

Parameter	Value
Tank cross-sectional area (S_c)	0.06 m ²
Leakage orifice area (a_1)	1.31×10^{-4} m ²
Leakage orifice area (a_2)	1.51×10^{-4} m ²
Leakage orifice area (a_3)	9.27×10^{-5} m ²
Leakage orifice area (a_4)	8.82×10^{-5} m ²
Three-way valve opening ratio (γ_a)	0.3
Three-way valve opening ratio (γ_b)	0.4
Gravitational acceleration (g)	9.81 m/s ²

trained on identical datasets according to learning Problem (30) where the parameter budget is ideally set to be approximately equal. However, for LSTM-MPC, to obtain an accurate model a higher amount of parameters were necessary, therefore the parameter budget was ignored. The trained models are implemented into the MPC scheme (3) which is constructed with input constraints $\mathcal{U} = \{u \in \mathbb{R}^2 | \text{col}(0,0) \leq u \leq \text{col}(4,4)\}$, tracking cost $Q = 100$ and input cost $R = 1$. Consider the control input and closed-loop output given in Figure 12, it can be seen that Mamba-MPC is able to track the piece-wise constant references for all system states and significantly outperforms LSTM-MPC whereas containing a significantly lower amount of parameters. Notably, the sampling time at 0.14s is well within the sampling time of the system at $T_s = 5s$ as can be seen in Table 4 and is significantly faster than the sampling time of LSTM-MPC at 0.65s.

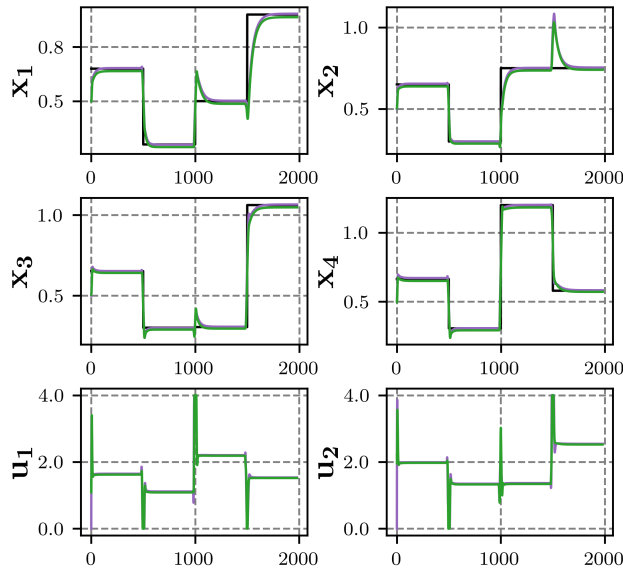
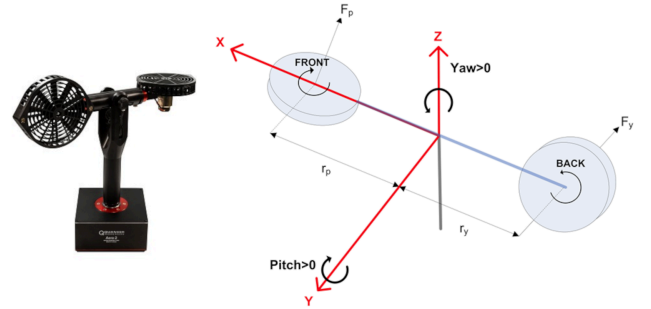

FIGURE 12: Closed-loop performance comparison between Mamba-MPC (—) and LSTM-MPC (—) on the MIMO Four Tank system for piece-wise constant reference trajectories (—).

TABLE 6: Parameters and Performance for Mamba-MPC on the Quanser Aero2 Setup.

Quanser Aero2	
Mamba	
<i>Model Architecture</i>	
Model Dim. D	8
Kernel Size	20
State Dim.	4
Horizon N	18
Layers	2
Params	983
<i>Controller Performance</i>	
MAE	$[87, 128, 134] \cdot 10^{-3}$
MSE	$[22, 90, 91] \cdot 10^{-3}$
Comp. Time (s)	0.038
Sampling Time (s)	0.04
<i>Training Metrics</i>	
Time (min)	400
Train Loss	$2.2 \cdot 10^{-3}$
Val. Loss	$2.0 \cdot 10^{-2}$


FIGURE 13: Left: Image of Quanser Aero2. Right: Free-body diagram of the dynamics of the Quanser Aero2

VII. Validation on a Physical Setup: The Quanser Aero2

To assess the performance of Mamba-MPC on a physical setup, the Quanser Aero2 is considered, which can be seen in Figure 13 alongside its free-body diagram. In this example, the 2-DOF configuration is considered. The Aero2 has inputs $u \in \mathbb{R}^2$ corresponding to the voltages supplied to the DC motors of rotor 1 and rotor 2. The system outputs $y \in \mathbb{R}^3$ encompassing the pitch angle, sine and cosine components of the yaw. For the initial conditions, pitch rate and yaw rate are added such that the initial condition $x_0 \in \mathbb{R}^5$. A dedicated training dataset is constructed via an open-loop experiment. Data acquisition is performed at a sampling frequency of $f_s = 100$ Hz for $T = 240 \cdot 10^3$ samples where the input signal is constructed with a multi-level *Pseudo Random Binary Signal* (PRBS) as carrier wave, with amplitudes $[\pm 8, \pm 10, \pm 12]$ and maximum switching frequency of 2 Hz. On top of the carrier wave, a multisine is superimposed featuring amplitudes 4, 2, 1, respectively, across a frequency range of 0.015 Hz to 2.485 Hz containing

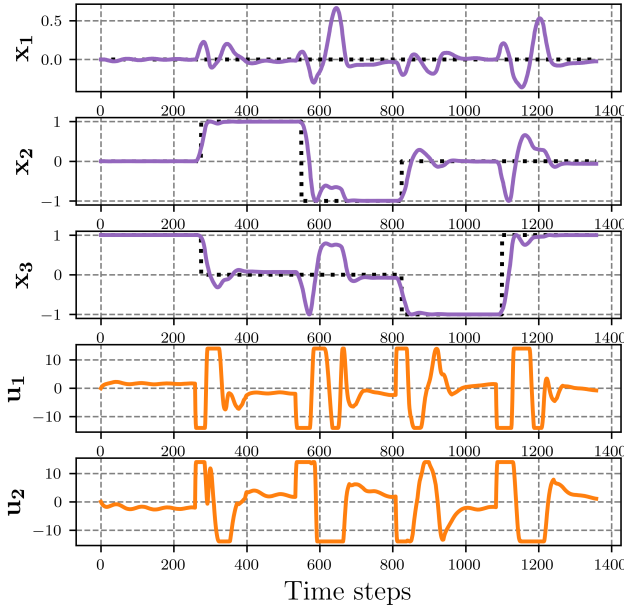


FIGURE 14: Closed-loop reference tracking of Mamba-MPC on the Quanser Aero2 physical setup. The states are indicated by (—) which track a piece-wise constant reference indicated by (---). The inputs are indicated by (—).

500 harmonics. Due to computational limitations, the data was down-sampled to $f_s = 25$ Hz to reduce the computational requirement. The definitive dataset is constructed according to (22) for a prediction horizon of $N = 18$. Mamba-MPC is trained for 2000 epochs according to (30) with hyperparameters given in Table 6.

The model is implemented into the MPC scheme (3) with $Q = \text{diag}(6, 1.5, 1.5)$, $R = \text{diag}(0.1, 0.1)$ and $P = 1 \cdot 10^3$ with input constraints $\mathcal{U} = \{u \in \mathbb{R}^2 | \text{col}(-14, -14) \leq u \leq \text{col}(14, 14)\}$ and pitch angle constraints $\mathcal{Y}_1 = \{x_1 \in \mathbb{R} | -0.7 \leq x_1 \leq 0.7\}$. To remain within the sampling frequency of $f_s = 25\text{Hz}$ an SQP solver is used in CasADI. To further improve the computational speed, a warm start of the Lagrangian multipliers is added. Consider the closed-loop response for a piece-wise constant reference signal in Figure 14. In general, Mamba-MPC is able to successfully track the reference signal for the pitch angle and yaw angle. In particular, the mean computation time per iteration is 0.038 s which is below the sampling time of the system at 0.04 s. Overall, Mamba-MPC is able to successfully stabilize and track a piece-wise constant reference on the Aero2 while remaining computationally tractable.¹

VIII. Conclusion

In this paper, we developed Mamba-MPC, an MPC formulation that utilizes Mamba NN as a multi-step predictor, and a novel data embedding for Seq2Seq modeling. A complete

¹A video demonstration of Mamba-MPC on the Quanser Aero2 is available at: <https://youtu.be/JZUzMLH1HeU>

and consistent mathematical description of the Mamba NN was given and the modeling advantages of Mamba were highlighted. We provided an embedding of initial conditions and input sequence such that decoder-only architectures can be utilized for multi-step prediction. We evaluated the performance of Mamba-MPC on two simulated nonlinear systems where we tested stabilization, noise robustness, reference tracking and MIMO reference tracking. Furthermore, we implemented Mamba-MPC on a physical Quanser Aero2 setup. On the simulated benchmarks, Mamba-MPC successfully stabilizes a nonlinear systems, it is robust to noisy data, outperforms LSTM-MPC on reference tracking, and scales favorably in comparison to LSTM-MPC and Transformer-MPC. Furthermore, on all benchmarks, Mamba-MPC stays within the desired computation time. Moreover, Mamba-MPC successfully performed MIMO reference tracking on the Quanser Aero2 at real-time control sampling interval of 0.04s. Future work will consider:

- Evaluating the performance of the SSM function when the diagonal terms in A are allowed to be complex. Complex valued diagonal terms could increase the modeling capacity of Mamba in dynamical systems;
- Implementing Mamba-MPC using *Acados* for further computational efficiency;
- Developing a closed-loop stability analysis framework for Mamba-MPC.

REFERENCES

- [1] T. Chen and H. Chen, “Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems,” *IEEE Transactions on Neural Networks*, vol. 6, pp. 911–917, 1995.
- [2] M. Lazar and O. Pastravanu, “A neural predictive controller for nonlinear systems,” *Mathematics and Computers in Simulation*, vol. 60, pp. 315–324, 2002.
- [3] Y. Pan and J. Wang, “Model predictive control for nonlinear affine systems based on the simplified dual neural network,” in *IEEE Conference on Control Applications (CCA) & Intelligent Control (ISIC)*, 2009, pp. 683–688.
- [4] J. H. Hoekstra, B. Cseppento, G. I. Beintema, M. Schoukens, Z. Kollr, and R. Tth, “Computationally efficient predictive control based on ANN state-space models,” in *Proceedings of the IEEE Conference on Decision and Control*. Institute of Electrical and Electronics Engineers Inc., 2023, pp. 6336–6341.
- [5] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, pp. 1735–1780, nov 1997.
- [6] N. Mohajerin and S. L. Waslander, “Multistep Prediction of Dynamic Systems With Recurrent Neural Networks,” vol. 30, no. 11, pp. 3370–3383. [Online]. Available: <https://ieeexplore.ieee.org/document/8630673/>
- [7] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning Phrase Representations using RNN EncoderDecoder for Statistical Machine Translation,” in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, pp. 1724–1734. [Online]. Available: <http://aclweb.org/anthology/D14-1179>
- [8] F. Bonassi, C. F. O. da Silva, and R. Scattolini, “Nonlinear MPC for offset-free tracking of systems learned by GRU neural networks,” 2021.
- [9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin, “Attention is All you Need.”
- [10] J. Park, M. R. Babaei, S. A. Munoz, A. N. Venkat, and J. D. Hedengren, “Simultaneous multistep Transformer architecture for model

- predictive control,” *Computers and Chemical Engineering*, vol. 178, oct 2023.
- [11] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” arXiv preprint arXiv:2312.00752, dec 2023. [Online]. Available: <http://arxiv.org/abs/2312.00752>
 - [12] A. Gu, K. Goel, and C. R., “Efficiently modeling long sequences with structured state spaces,” arXiv preprint arXiv:2111.00396, oct 2021. [Online]. Available: <http://arxiv.org/abs/2111.00396>
 - [13] C. Tallec and Y. Ollivier, “Can recurrent neural networks warp time?” arXiv preprint arXiv:1804.11188, mar 2018. [Online]. Available: <http://arxiv.org/abs/1804.11188>
 - [14] L. Lu, P. Jin, and G. E. Karniadakis, “DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators,” *Nature Machine Intelligence*, vol. 3, pp. 218–229, mar 2021.
 - [15] Z. Hu, N. A. Daryakenari, Q. Shen, K. Kawaguchi, and G. E. Karniadakis, “State-Space Models are Accurate and Efficient Neural Operators for Dynamical Systems,” [Online]. Available: <https://www.ssrn.com/abstract=4990229>
 - [16] Z. Li, M. Han, and X. Yin, “MamKO: Mamba-based koopman operator for modeling and predictive control,” in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=hNjCVVm0EQ>
 - [17] Y. Matsui and T. Yokota, “Broadcast Product: Redefining Shape-aligned Element-wise Multiplication and Beyond,” Jun. 2026.
 - [18] T. O. De Jong, K. Shukla, and M. Lazar, “Deep Operator Neural Network Model Predictive Control,” vol. 4, pp. 501–517. [Online]. Available: <https://ieeexplore.ieee.org/document/11181185/>
 - [19] N. Shazeer, “GLU variants improve Transformer,” arXiv preprint arXiv:2002.05202, feb 2020. [Online]. Available: <http://arxiv.org/abs/2002.05202>
 - [20] M. Lezcano-Casado and D. Martinez-Rubio, “Cheap orthogonal constraints in neural networks: A simple parametrization of the orthogonal and unitary group,” arXiv preprint arXiv:1901.08428, may 2019. [Online]. Available: <http://arxiv.org/abs/1901.08428>
 - [21] E. S. Lubana, R. P. Dick, and H. Tanaka, “Beyond BatchNorm: Towards a unified understanding of normalization in deep learning,” arXiv preprint arXiv:2106.05956, oct 2021. [Online]. Available: <http://arxiv.org/abs/2106.05956>
 - [22] B. Zhang and R. Sennrich, “Root Mean Square Layer Normalization,” [Online]. Available: <https://www.zora.uzh.ch/id/eprint/177483>
 - [23] L. Bold, I. Schimperna, K. Worthmann, and J. Khler, “Exponential stability of data-driven nonlinear mpc based on input/output models,” *IEEE Control Systems Letters*, vol. 10, pp. 511–516, 2026.
 - [24] H. Sun, Z. Wu, B. Xia, P. Chang, Z. Dong, Y. Yuan, Y. Chang, and X. Wang, “Entropy-based activation function optimization: a method on searching better activation functions,” in *The Thirteenth International Conference on Learning Representations*, 2025.
 - [25] R. Luo, “Principles of lipschitz continuity in neural networks,” arXiv preprint arXiv:2602.04078, 2026.
 - [26] L. Cao, W. Liu, Z. Chen, and Y. Qin, “A control-theoretic view of mamba on stability and robustness,” in *Forty-third International Conference on Machine Learning*, 2026. [Online]. Available: <https://openreview.net/forum?id=kLfm50q9kO>
 - [27] M. awryczuk and K. Zarzycki, “LSTM and GRU type recurrent neural networks in model predictive control: A review,” *Neurocomputing*, vol. 632, jun 2025.
 - [28] S. M. Emadi, “Exact Attention Sensitivity and the Geometry of Transformer Stability,” Feb. 2026, arXiv:2602.18849 [cs.LG]. [Online]. Available: <http://arxiv.org/abs/2602.18849>
 - [29] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” [Online]. Available: <http://arxiv.org/abs/1412.6980>
 - [30] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, pp. 25–57, may 2006.
 - [31] P. C. Blaud, P. Chevrel, F. Claveau, P. Haurant, and A. Mouraud, “Four MPC implementations compared on the Quadruple Tank Process benchmark: pros and cons of neural MPC,” in *IFAC-PapersOnLine*, vol. 55. Elsevier B.V., jul 2022, pp. 344–349.



versarial conditions.



Technology, Eindhoven, The Netherlands.



mechatronics, automotive and biological systems. Dr. Lazar received the European Embedded Control Institute Ph.D. Award in 2007 for his Ph.D. dissertation. He received a VENI personal grant from the Dutch Research Council (NWO) in 2008 and he supervised 13 Ph.D. researchers (2 received the Cum Laude distinction). Dr. Lazar chaired the 4th IFAC Conference on Nonlinear Model Predictive Control in 2012. He is an Active Member of the IFAC Technical Committees 1.3 Discrete Event and Hybrid Systems, 2.3 Nonlinear Control Systems and an Associate Editor of IEEE Transactions on Automatic Control.

Michiel Cevaal received the B.S. degree in Electrical Engineering from Eindhoven University of Technology, Eindhoven, The Netherlands, in 2022 and the M.S. degree in Electrical Engineering from Eindhoven University of Technology, Eindhoven, The Netherlands in 2025. He is currently pursuing the Ph.D. degree at Tilburg University in collaboration with the Data Science Center of Excellence of the Ministry of Defense. His research is focused on autonomous multi-agent drone swarms in ad-

Thomas O. de Jong (Student Member, IEEE) received the B.S. degree in Mechanical Engineering from Eindhoven University of Technology, Eindhoven, The Netherlands, in 2021 and the M.S. degree in Systems and Control from Eindhoven University of Technology, Eindhoven, The Netherlands, in 2023. He is currently pursuing the Ph.D. degree in Systems and Control in Eindhoven, The Netherlands, in 2021 and the M.S. degree in Systems and Control at Eindhoven University of Technology, Eindhoven, The Netherlands.

Dr. Mircea Lazar (Senior Member, IEEE) is a Full Professor in Constrained Control of Complex Systems at the Electrical Engineering Department, Eindhoven University of Technology, The Netherlands. His research interests cover Lyapunov functions, distributed control, neural networks and constrained control of nonlinear and hybrid systems, including model predictive control. His research is driven by control problems in power systems, power electronics, high-precision mechatronics, automotive and biological systems. Dr. Lazar received the European Embedded Control Institute Ph.D. Award in 2007 for his Ph.D. dissertation. He received a VENI personal grant from the Dutch Research Council (NWO) in 2008 and he supervised 13 Ph.D. researchers (2 received the Cum Laude distinction). Dr. Lazar chaired the 4th IFAC Conference on Nonlinear Model Predictive Control in 2012. He is an Active Member of the IFAC Technical Committees 1.3 Discrete Event and Hybrid Systems, 2.3 Nonlinear Control Systems and an Associate Editor of IEEE Transactions on Automatic Control.